Computational Programming with Python

Lecture 2: Lists and elementary plotting

Center for Mathematical Sciences, Lund University

Lecturer: Malin Christersson, Robert Klöfkorn

This lecture

- Intro Survey
- Use of AI in Programming
- Lists
- More about for loops
- Elementary plotting

Use of AI in Programming

- It is increasingly common for people to use generative AI chatbots such as ChatGPT to get help while coding.
- Answers found on `stackoverflow` vs answers by `AI based chatbots` based on advanced statistical models generating the **most likely** sequence of text that would follow the prompt they are given.
- You need the knowledge and skills to be able to understand these responses, to judge whether or not they are accurate, and to fix any errors in the code it offers you. **This course teaches these skills**.

We recommend that you **avoid getting help from generative AI during this course** for several reasons:

1. For most problems you will encounter at this stage, help and answers can be found among the first results returned by searching the internet.
2. The foundational knowledge and skills you will learn in this lesson by writing and fixing your own programs are essential to be able to evaluate the

correctness and safety of any code you receive from online help or a generative AI chatbot. Having solid knowledge in programming will help you to use such tools more effectively in the future.

3. As you start out with programming, the mistakes you make will be the kinds that have also been made – and overcome! – by everybody else who learned to program before you.

See also the primer on **AI Environmental Impact**.

Lists

A Python list is an ordered list of objects, enclosed in square brackets.
One accesses elements of a list using **zero-based** indices inside square brackets.
Note: `list` is the most commonly used container in Python.

List examples

In [ ]:

```
L1 = [1, 2]
print(L1[0])
print(L1[1])
print(type(L1))
# print(L1[2]) # raises IndexError
#print(L1[2])
```

In [ ]:

```
L2 = ['a', 1, [3, 4]]
print(L2[0])
print(L2[2])
print(L2[2][0])
```

Negative indices

In [ ]:

```
L2 = ['a', 1, [3, 4]]
print(L2[-1]) # last element
# print(L2[len(L2)-1])
print(L2[-2]) # second last element
# print(L2[-4])
```

List utilities - `range`

`range(n)` can be used to fill a list with n elements starting with zero.

In [ ]:

```
L3 = list(range(5)) # numbers 0,1,2,3,4
print(L3)
```

List utilities - `len`

The function `len(L)` gives the the length of a list:

In [ ]:

```
L4 = ["Stockholm", "Paris", "Berlin"]
L5 = [[0,1], [10, 100, 1000, 10000]]
print(len(L4))
print(len(L5))
print("len(L5[-1])", len(L5[-1]))

print(len(['a', 'b', 'c', 'd', 'e']))
print(L5[0][0])
# print(len(L5[0][0])) # gives TypeError
```

List utilities - `append`

Use the method `append` to append an element at the end of the list.

In [ ]:

```
L6 = ['a', 'b', 'c']
# L6 = []
print(L6[-1])
print(len(L6))
#L6.append('d')
L6[ 0 ] = 'd'
print(L6[0])
print(L6[-1])
print(len(L6))
```

List comprehension

A convenient way to build up lists is to use the *list comprehension* construct, possibly with a conditional inside.

Definition

The syntax of a list comprehension is

```
[<expr> for <variable> in <list>]
```

Example:

In [ ]:

```
L1 = [2, 3, 10, 1, 5]
L2 = [x*2 for x in L1]
L3 = [x*2 for x in L1 if 4 < x <= 10]

print(L1)
print(L2)
print(L3)
```

List comprehension in mathematics

Mathematical notation

This is very close to the mathematical notation for sets. Compare:

$$L_2 = \{2x;\ x \in L_1\}$$

with

```
L2 = [2*x for x in L1]
```

Operations on lists

Adding two lists *concatenates* (sammanfogar) them:

In [ ]:

```
L1 = [1, 2]
L2 = [3, 4]
L3 = L1 + L2
print(L3)
```

More operations on lists

Logically, multiplying a list with an integer concatenates the list with itself several times: `n*L` is equivalent to

$$\underbrace{L + L + \cdots + L}_{n \text{ times}}$$

In [ ]:

```
L = [1, 2]
L2 = 3*L
print(L2)
```

Some more functions

Given a list `lst`:

- `sum(lst)`
- `max(lst)`
- `min(lst)`

In [ ]:

```
L = [4, 9, -1, 2]
print("The sum is", sum(L))
print("The minimum value is", min(L))
print("The maximum value is",max(L))
```

Min and max for strings

When comparing strings with the `>` or `<` operation then first the leading character is compared, which follows the ordering in the **UTF-8**.

In [ ]:

```
L = ['!adbg', 'Berlin', 'Berl']
print(min(L))
print(max(L))

print(len(L))
```

Some more methods

Given a list `lst`:

- `lst.reverse()`
- `lst.sort()`
- `lst.sort(reverse=True|False, key=myFunc)`

In [ ]:

```
L = [4, 9, -1, 2]
L3 = L.copy()
print("L =", L)
L.reverse()
print("L =", L)
L.sort()
print("L =", L)

L.insert(100, 8)
print(L)
```

Programming example

How do we calculate

$$\sum_{i=0}^{n} i$$

using a for loop? using a list? **5min, let's go!**

In [ ]:

```

```

In [ ]:

```
s = 0
n = 10
for i in range(n+1):
    s += i
print(s)
```

In [ ]:

```
s = sum([i for i in range(n+1)])

s = sum(range(n+1))
print(s)
```

Slicing and other list tools

Enumerating lists

Sometimes it is helpful to `enumerate` while iterating over elements of a list.

In [ ]:

```
L = ['C', 'l', 'o', 'u', 'd']
i = 0
for l in L:
    print(f"The {i}-th element of L is '{l}'")
    i += 1
#for i in range(len(L)):
#    print(f"The {i}-th element of L is '{L[i]}'")

for i, l in enumerate(L):
    print(f"The {i}-th element of L is '{l}'")
```

List access (repeat)

We can access elements of a list by using square brackets.

In [ ]:

```
L = ['C', 'l', 'o', 'u', 'd']
print(L[0])
print(L[-1])
```

In a similar way we can access sublists.

Creating sublists - Slicing

Slicing a list between index i and j is forming a new list by taking elements starting at i and ending *just before* j.

Example

In [ ]:

```
L = ['C', 'l', 'o', 'u', 'd']
print(L[1:4])
```

Compare with `range`.

Partial slicing

One may omit the first or the last bound of the slicing:

In [ ]:

```
L = ['C', 'l', 'o', 'u', 'd']
print("L[1:] =", L[1:])     # to the end
print("L[:3] =", L[:3])     # from the start
print("L[-2:] =", L[-2:])   # negative index
print("L[:-2] =", L[0:-2])
```

Mathematical analogy

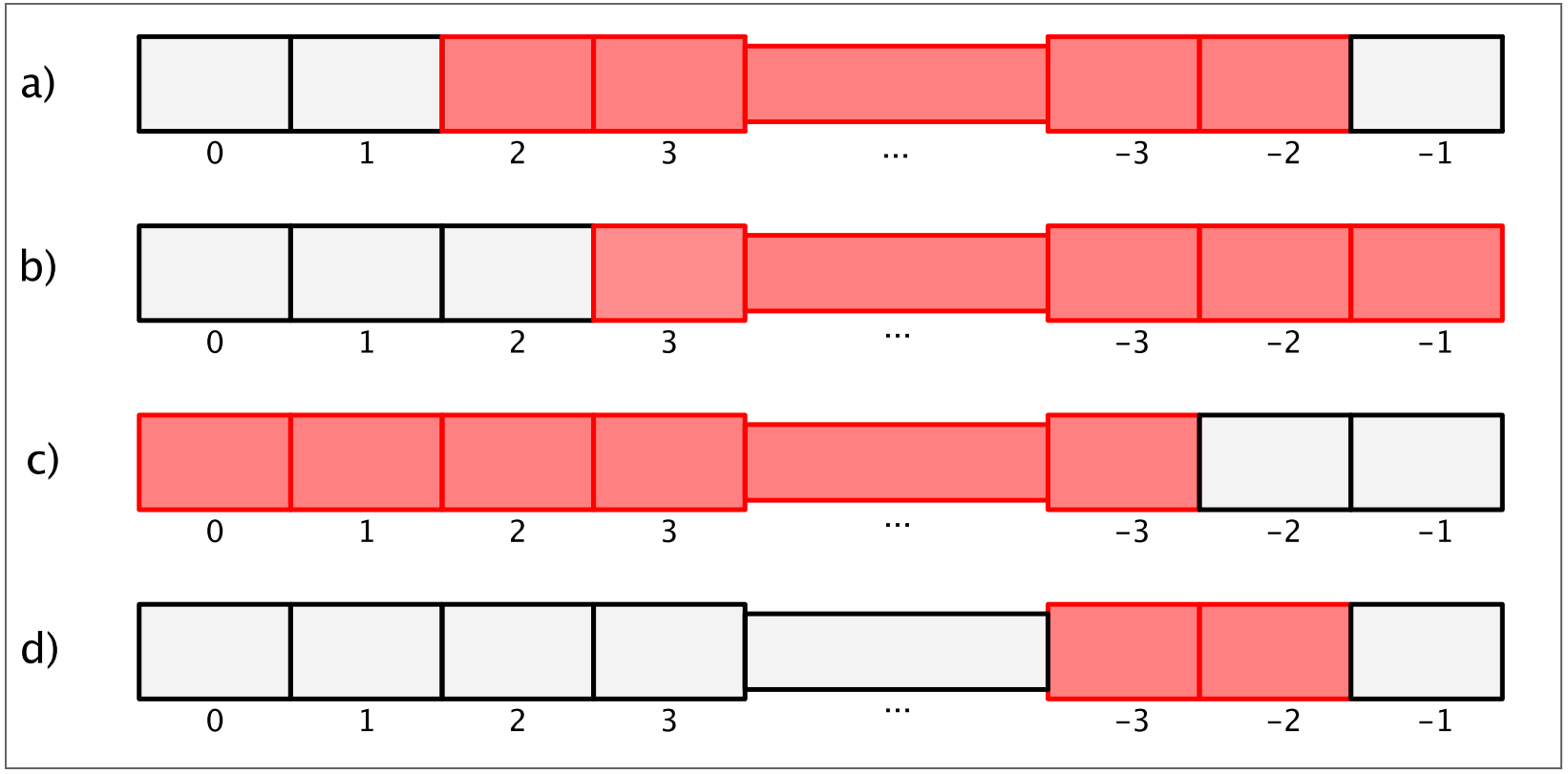
The interval $[-\infty, a)$ means: take all numbers strictly lower than a, cf. syntax of `L[:j]`.

To sum it up

- `L[i:]` take all elements *except* the i first ones
- `L[:i]` take the first i elements
- `L[-i:]` take the last i elements
- `L[:-i]` take all elements *except* the i last ones

Examples

From a list `L`, create the red sublists. **5 min, let's go!**



In [ ]:

In [ ]:

```
# Results
L = list(range(10))
print(L)

# a)
L1 = L[2:-1]
print(L1)

# b)
L2 = L[3:]
print(L2)

# c)
L3 = L[:-2]
print(L3)

# d)
L4 = L[-3:-1]
print(L4)
```

for loops (revisited)

A `for` loop allows to loop through a list using an index variable. This variable takes succesively the values of the elements in the list.
Example:

In [ ]:

```
L = [1, 2, 10]
for s in L:
    print(s * 2)
    print("Hello", s)

print("Loop finished")
```

Repeating a task

One typical use of the for loop is to repeat a certain task a fixed number of times:

In [ ]:

```
def do_something(i):
    print(i)

n = 10
for i in range(n):
    do_something(i) # this gets executed n times
```

Indentation

The part to be repeated in the for loop has to be properly *indented*.

```
for elt in my_list:
    do_something()
    something_else()
    etc
print("loop finished") # outside of the for loop
```

In contrast to other programming languages, the indentation in Python is **mandatory**.
Many other kinds of Python structures also have to be indented and this will be covered when introduced.

The full `for` statement - `break`

`break` gets out of the for loop before all elements of the list are used.
In the following example we possibly don't use all values of the list `x_values`.

In [ ]:

```
x_values=[0,2,3,4,5]
threshold=3.5
for x in x_values:
    if x > threshold:
        break
    print(x)
```

Not all entries were visited. **We say, the list was not exhausted.**

The full `for` statement - `else`

`else` checks if the loop was not *broken* by the `break` keyword.

In [ ]:

```
threshold=3.5
for x in x_values:
    if x > threshold:
        print("not all x are below the threshold")
        break
    print(x)
else:
    print("all the x are below the threshold")
```

If we did not break, the else-block is executed.

Elementary plotting

We must first make the central visualization tool in Python available:

In [ ]:

```
from numpy import *
from matplotlib.pyplot import *
```

Then we generate two lists:

In [ ]:

```
x_list = list(range(100))
y_list = [sqrt(x) for x in x_list]
```

Then making a graph is simple:

In [ ]:

```
# from above
from matplotlib.pyplot import *
from numpy import *
x_list = list(range(100))
y_list = [sqrt(x) for x in x_list]
# plotting commands
plot(x_list, y_list, '-')
title('My first plot')
xlabel('x')
ylabel('Square root of x')
show() # not always needed
```

More about plots

- You have already used the command `plot`. It needs a list of x values and a list of y values. If a single list, y is given, the list `list(range(len(y)))` is assumed as x values.
- You may use the keyword argument `label` to give your curves a name, and then show them using `legend`.

In [ ]:

```
from numpy import *
from matplotlib.pyplot import *
x_vals = [.2*n for n in range(20)]
y1 = [sin(.3*x) for x in x_vals]
y2 = [sin(2*x) for x in x_vals]
plot(x_vals ,y1, label='sin(0.3*x)')
plot(x_vals, y2, label='sin(2*x)')
xlabel('x')
ylabel('sin...')
legend()
show() # not always needed, but does not hurt to put it here
```

A plot with more key words

Here is an example with more keywords:

In [ ]:

```
plot(x_vals, y2,
     color='green',
     linestyle='dashed',
     marker='o',
     markerfacecolor='blue',
     markersize=12,
     linewidth=6)
```